

Experiment No.5
Study and Implementation of Lexical Analyzer.
Date of Performance:
Date of Submission:



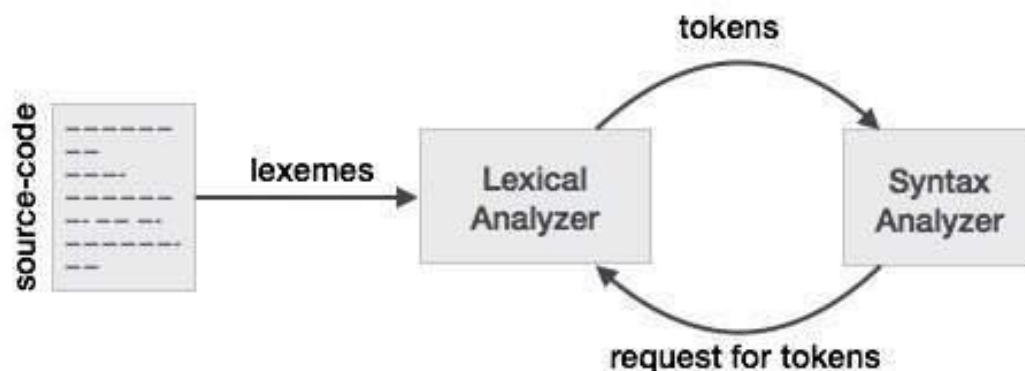
Aim: Study and Implementation of Lexical Analyzer.

Objective: To study and implement lexical analyzer program for identifying tokens from given input source program.

Theory:

Lexical analysis is the first phase of a compiler. It takes the modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code.

If the lexical analyzer finds a token invalid, it generates an error. The lexical analyzer works closely with the syntax analyzer. It reads character streams from the source code, checks for legal tokens, and passes the data to the syntax analyzer when it demands.



Lexical analysis : process of taking an input string of characters (such as the source code of a computer program) and producing a sequence of symbols called lexical tokens, or just tokens, which may be handled more easily by a parser. The pattern matches each string in the set.

Tokens:

Lexemes are said to be a sequence of characters (alphanumeric) in a token. There are some predefined rules for every lexeme to be identified as a valid token. These rules are defined by grammar rules, by means of a pattern. A pattern explains what a token can be, and these patterns are defined by means of regular expressions.



In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

For example, in C language, the variable declaration line

```
int value = 100;
```

contains the tokens:

int (keyword), value (identifier), = (operator), 100 (constant) and; (symbol).

Specifications of Tokens:

Let us understand how the language theory undertakes the following terms:

Alphabets

Any finite set of symbols $\{0,1\}$ is a set of binary alphabets,

$\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$ is a set of Hexadecimal alphabets, $\{a-z, A-Z\}$ is a set of English language alphabets.

Strings

Any finite sequence of alphabets is called a string. Length of the string is the total number of occurrence of alphabets, e.g., the length of the string tutorialspoint is 14 and is denoted by $|\text{tutorialspoint}| = 14$. A string having no alphabets, i.e. a string of zero length is known as an empty string and is denoted by ϵ (epsilon).

Special Symbols

A typical high-level language contains the following symbols:-

Arithmetic Symbols	Addition(+), Subtraction(-), Modulo(%), Multiplication(*), Division(/)
Punctuation	Comma(,), Semicolon(;), Dot(.), Arrow(->)
Assignment	=
Special Assignment	+=, /=, *=, -=
Comparison	==, !=, <, <=, >, >=
Preprocessor	#
Location Specifier	&
Logical	&, &&, , , !
Shift Operator	>>, >>>, <<, <<<



Language

A language is considered as a finite set of strings over some finite set of alphabets. Computer languages are considered as finite sets, and mathematically set operations can be performed on them. Finite languages can be described by means of regular expressions.

Code:

```
// C program to illustrate the implementation of lexical
```

```
// analyser
```

```
#include <ctype.h>
```

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_LENGTH 100
```

```
// this function check for a delimiter(it is a piece of data
```

```
// that separated it from other) to perform some specific case
```

```
// on it
```

```
bool isDelimiter(char chr)
```

```
{
```

```
    return (chr == ' ' || chr == '+' || chr == '-'
```

```
            || chr == '*' || chr == '/' || chr == ','
```

```
            || chr == ';' || chr == '%' || chr == '>'
```

```
            || chr == '<' || chr == '=' || chr == '('
```

```
            || chr == ')' || chr == '[' || chr == ']'
```

```
            || chr == '{' || chr == '}');
```

```
CSL601: System Programming and Compiler Construction Laboratory
```



```
}
```

```
// this function check for a valid identifier eg:- +,-* etc
```

```
bool isOperator(char chr)
```

```
{
```

```
    return (chr == '+' || chr == '-' || chr == '*'
```

```
            || chr == '/' || chr == '>' || chr == '<'
```

```
            || chr == '=');
```

```
}
```

```
// this function check for an valid identifier
```

```
bool isValidIdentifier(char* str)
```

```
{
```

```
    return (str[0] != '0' && str[0] != '1' && str[0] != '2'
```

```
            && str[0] != '3' && str[0] != '4'
```

```
            && str[0] != '5' && str[0] != '6'
```

```
            && str[0] != '7' && str[0] != '8'
```

```
            && str[0] != '9' && !isDelimiter(str[0]));
```

```
}
```

```
// 32 Keywords are checked in this function and return the
```

```
// result accordingly
```

```
bool isKeyword(char* str)
```

```
{
```

```
    const char* keywords[]
```

```
        = { "auto",      "break", "case",      "char",
```

```
            "const", "continue", "default", "do",
```

```
            "double", "else",      "enum",      "extern",
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
"float", "for",    "goto", "if",
"int",    "long", "register", "return",
"short", "signed", "sizeof", "static",
"struct", "switch", "typedef", "union",
"unsigned", "void",    "volatile", "while" };

for (int i = 0;
    i < sizeof(keywords) / sizeof(keywords[0]); i++) {
    if (strcmp(str, keywords[i]) == 0) {
        return true;
    }
}

return false;
}

// check for an integer value
bool isInteger(char* str)
{
    if (str == NULL || *str == '\0') {
        return false;
    }

    int i = 0;
    while (isdigit(str[i])) {
        i++;
    }

    return str[i] == '\0';
}

// trims a substring from a given string's start and end
```



```
// position
char* getSubstring(char* str, int start, int end)
{
    int length = strlen(str);
    int subLength = end - start + 1;
    char* subStr
        = (char*)malloc((subLength + 1) * sizeof(char));
    strncpy(subStr, str + start, subLength);
    subStr[subLength] = '\0';
    return subStr;
}

// this function parse the input
int lexicalAnalyzer(char* input)
{
    int left = 0, right = 0;
    int len = strlen(input);

    while (right <= len && left <= right) {
        if (!isDelimiter(input[right]))
            right++;

        if (isDelimiter(input[right]) && left == right) {
            if (isOperator(input[right]))
                printf("Token: Operator, Value: %c\n",
                    input[right]);

            right++;
        }
    }
}
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
        left = right;
    }
    else if (isDelimiter(input[right]) && left != right
        || (right == len && left != right)) {
        char* subStr
            = getSubstring(input, left, right - 1);

        if (isKeyword(subStr))
            printf("Token: Keyword, Value: %s\n",
                subStr);

        else if (isInteger(subStr))
            printf("Token: Integer, Value: %s\n",
                subStr);

        else if (isValidIdentifier(subStr)
            && !isDelimiter(input[right - 1]))
            printf("Token: Identifier, Value: %s\n",
                subStr);

        else if (!isValidIdentifier(subStr)
            && !isDelimiter(input[right - 1]))
            printf("Token: Unidentified, Value: %s\n",
                subStr);

        left = right;
    }
}

return 0;
```




```
}

// main function
int main()
{
    // Input 01
    char lex_input[MAX_LENGTH] = "int a = b + c";
    printf("For Expression \"%s\\\":\\n", lex_input);
    lexicalAnalyzer(lex_input);
    printf(" \\n");
    // Input 02
    char lex_input01[MAX_LENGTH]
        = "int x=ab+bc+30+switch+ 0y ";
    printf("For Expression \"%s\\\":\\n", lex_input01);
    lexicalAnalyzer(lex_input01);
    return (0);
}
```

Output:

For Expression 1:

Token: Keyword, Value: int

Token: Identifier, Value: a

Token: Operator, Value: =

Token: Identifier, Value: b

Token: Operator, Value: +

Token: Identifier, Value: c



For Expression 2:

Token: Keyword, Value: int

Token: Identifier, Value: x

Token: Operator, Value: =

Token: Identifier, Value: ab

Token: Operator, Value: +

Token: Identifier, Value: bc

Token: Operator, Value: +

Token: Integer, Value: 30

Token: Operator, Value: +

Token: Keyword, Value: switch

Token: Operator, Value: +

Token: Unidentified, Value: 0y

Conclusion:

The above program is an implementation of a lexical analyzer program in C language. In the compilation process, the Lexical analysis phase is the first step. In this step, the lexical analyzer breaks down the input code into small units called tokens (for example keywords, identifiers, operators, literals, and punctuation).